

루프 엔지니어링: 루프 시작하기

Loop engineering: Getting started with loops

카테고리: Claude Code

제품: Claude Code

작성일: 2026년 6월 30일

읽는 시간: 약 5분

원문: <https://claude.com/blog/getting-started-with-loops>

요즘 코딩 에이전트에게 프롬프트를 주는 대신 "루프를 설계한다"거나 "루프 엔지니어링"이라는 말이 자주 들립니다. X(트위터)에서 '루프'가 정확히 무엇인지 찾아보면 저마다 다른 답을 내놓는 것을 발견하게 됩니다.

Claude Code 팀에서는 루프를 "에이전트가 정지 조건이 충족될 때까지 작업 사이클을 반복하는 것"으로 정의합니다. 저희는 루프를 다음 기준에 따라 몇 가지 유형으로 분류합니다:

- 어떻게 트리거되는가
- 어떻게 멈추는가
- 어떤 Claude Code 기본 기능(primitive)이 사용되는가
- 각 유형에 가장 적합한 작업 종류는 무엇인가

이 글에서는 주요 루프 유형들과 각각을 언제 사용해야 하는지, 그리고 토큰 사용량을 관리하면서 코드 품질을 유지하는 방법을 다룹니다. 모든 작업에 복잡한 루프가 필요한 것은 아닙니다. 가장 단순한 해법부터 시작해서 이러한 패턴들을 선택적으로 사용하세요.

턴 기반 루프 (Turn-based loops)

- **트리거:** 사용자 프롬프트
- **정지 조건:** Claude 가 작업을 완료했다고 판단하거나 추가 컨텍스트가 필요하다고 판단할 때
- **가장 적합한 용도:** 정기적인 프로세스나 일정에 속하지 않는 짧은 작업

● **관리 방법:** 구체적인 프롬프트를 작성하고, 스킬을 활용해 검증을 개선함으로써
턴 수를 줄이기

사용자가 보내는 모든 프롬프트는 각 턴을 사용자가 직접 지시하는 수동 루프를 시작시킵니다. Claude는 컨텍스트를 수집하고, 작업을 수행하고, 결과를 확인하고, 필요하면 반복한 뒤 응답합니다. 이를 '에이전틱 루프(agentic loop)'라고 부릅니다.

예를 들어 Claude에게 좋아요 버튼을 만들어 달라고 요청하면, Claude는 코드를 읽고, 수정하고, 테스트를 실행한 뒤 스스로 동작한다고 믿는 결과물을 넘겨줍니다. 그러면 사용자가 직접 결과를 확인하고 다음 프롬프트를 작성하게 됩니다.

수동으로 확인하던 단계를 SKILL.md 파일로 인코딩하면 Claude가 스스로 더 많은 작업을 엔드투엔드로 검증할 수 있게 되어 이 검증 단계를 개선할 수 있습니다. (스킬, 혹, 서브에이전트 중 이러한 자동화에 무엇을 선택할지는 Claude Code 운영 가이드를 참고하세요.)

여기에는 Claude가 결과물을 직접 확인하거나, 측정하거나, 상호작용할 수 있는 도구나 커넥터가 포함되어야 합니다. 점검 항목이 정량적일수록 Claude가 스스로 검증하기가 쉬워집니다.

예를 들어 SKILL.md 파일에 다음과 같이 명시할 수 있습니다:

```
---
name: verify-frontend-change
description: Verify any UI change end-to-end before declaring it done.
---

# 프론트엔드 변경 사항 검증하기
UI 변경 사항을 편집 성공만으로 완료라고 보고하지 마세요. 사람 리뷰어처럼
검증하세요:

1. 개발 서버를 시작하고 편집한 페이지를 브라우저에서 엽니다.

2. 변경 사항과 직접 상호작용합니다. 새로운 컨트롤(버튼, 입력창, 토글)의 경우:
   클릭해 보고, 예상된 상태 변화를 확인하고, 변경 전/후를 스크린샷으로 남깁니다.

3. 브라우저 콘솔을 확인합니다: 새로운 오류나 경고가 없어야 합니다.
```

4. Chrome Devtools MCP를 사용해 성능 트레이스를 실행하고 Core Web Vitals를 점검합니다.

어느 단계라도 실패하면 문제를 수정하고 1단계부터 다시 실행하세요.
일부만 검증된 작업을 넘기지 마세요.

목표 기반 루프 (Goal-based loop, /goal)

- **트리거:** 실시간 수동 프롬프트
- **정지 조건:** 목표 달성 또는 최대 턴 수 도달
- **가장 적합한 용도:** 검증 가능한 종료 기준이 있는 작업
- **관리 방법:** 구체적인 완료 기준과 명시적인 턴 상한을 설정하기(예: "5 번 시도 후 중단")

한 번의 턴만으로는 부족할 때가 있습니다. 특히 더 복잡한 작업일수록 그렇습니다. 에이전트는 반복할 수 있을 때 더 나은 성과를 냅니다. /goal을 사용해 '완료'가 무엇인지 정의하면 Claude가 계속 반복하는 시간을 늘릴 수 있습니다.

성공 기준을 정의하면 Claude가 '충분히 잘 됐다'는 판단을 스스로 내려 루프를 조기에 종료할 필요가 없어집니다. Claude가 멈추려고 시도할 때마다 평가자 모델이 설정한 조건을 확인하고, 목표가 달성되거나 정해진 턴 수에 도달할 때까지 다시 작업을 시킵니다.

이 때문에 통과한 테스트 수나 특정 점수 임계값 달성처럼 결정론적인 기준이 매우 효과적입니다.

예를 들면:

/goal 홈페이지 Lighthouse 점수를 90점 이상으로 올려줘. 5번 시도 후 중단.

시간 기반 루프 (Time-based loop, /loop 및 /schedule)

- **트리거:** 지정된 시간 간격
- **정지 조건:** 사용자가 취소하거나 작업이 완료될 때(PR 이 머지되거나 큐가 비었을 때)
- **가장 적합한 용도:** 반복적인 작업, 또는 외부 환경/시스템과의 연동
- **관리 방법:** 간격을 더 길게 설정하거나, 시간이 아닌 이벤트 기반으로 반응하도록 하기

일부 에이전트 작업은 반복적입니다. 작업 자체는 동일하고 입력값만 바뀝니다. 예를 들어 매일 아침 Slack 메시지를 요약하는 작업이 그렇습니다. 다른 작업은 외부 시스템에 의존하는데, 이런 시스템과 연동하는 간단한 방법은 일정 간격으로 상태를 확인하고 변경 사항에 반응하는 것입니다. 예를 들어 코드 리뷰를 받거나 CI가 실패할 수 있는 PR이 여기에 해당합니다.

이런 경우에는 `/loop`로 Claude 실행을 트리거할 수 있으며, 이는 지정한 간격으로 프롬프트를 재실행합니다. 예를 들면:

```
/loop 5분마다 내 PR을 확인하고, 리뷰 코멘트에 대응하고, 실패한 CI를 수정해줘
```

`/loop`는 사용자의 컴퓨터에서 실행되므로 컴퓨터를 끄면 중지됩니다. `/schedule`로 루틴을 만들면 이 루프를 클라우드로 옮길 수 있습니다.

능동형 루프 (Proactive loops)

- **트리거:** 실시간 사람의 개입 없이, 이벤트 또는 일정
- **정지 조건:** 각 작업은 목표가 달성되면 종료됩니다. 루틴 자체는 사용자가 끝 때까지 계속 실행됩니다.
- **가장 적합한 용도:** 버그 리포트, 이슈 트리아지, 마이그레이션, 의존성 업그레이드 등 명확하게 정의된 반복 작업 흐름
- **관리 방법:** 루틴 작업은 더 작고 빠른 모델로 라우팅하고, 판단이 필요한 작업에는 가장 성능 좋은 모델을 사용하기

위에서 소개한 기본 기능들은 auto 모드, 동적 워크플로(연구 프리뷰) 같은 다른 Claude Code 기능들과 결합되어 장시간 실행되는 작업을 위한 루프로 구성될 수 있습니다.

예를 들어 들어오는 피드백을 처리하려면 다음을 사용할 수 있습니다:

- **1. `/schedule`**(연구 프리뷰) — 새 리포트를 확인하는 루틴 실행
- **2. `/goal`**로 완료 상태를 정의하고, 스킬로 검증 방법을 문서화
- **3. 동적 워크플로**로 각 리포트를 분류(triage)하고, 수정하고, 수정 사항을 리뷰하는 에이전트들을 오케스트레이션
- **4. Auto 모드**를 사용해 권한을 묻기 위해 멈추지 않고 루틴이 실행되도록 함

이를 종합하면 프롬프트는 다음과 같은 형태가 될 수 있습니다:

```
/schedule 매시간: #project-feedback 채널에서 버그 리포트를 확인해줘.  
/goal: 이번 실행에서 발견된 모든 리포트가 분류(triage)되고, 처리되고,
```

응답될 때까지 멈추지 마. 버그를 수정할 때는 워크플로를 사용해 3개의 병렬 워크트리에서 해결책을 탐색하고, 심사자(judge)가 이를 적대적으로(adversarially) 검토하게 해줘.

코드 품질 유지하기

루프가 만들어내는 결과물의 품질은 그 루프를 둘러싼 시스템에 달려 있습니다. 시스템을 설계할 때 다음을 고려하세요:

- **코드베이스 자체를 깔끔하게 유지하세요:** Claude 는 코드베이스에 이미 존재하는 패턴과 관례를 따릅니다.
- **Claude 에게 스스로 작업을 검증할 방법을 제공하세요:** 스킬을 이용해 사용자와 팀에게 '좋은 결과'가 무엇인지 인코딩하세요.
- **문서에 쉽게 접근할 수 있게 하세요:** 프레임워크와 라이브러리 문서에는 최신 모범 사례가 담겨 있습니다.
- **코드 리뷰에는 별도의 에이전트를 사용하세요:** 새로운 컨텍스트를 가진 리뷰어는 편향이 적고, 메인 에이전트의 추론에 영향을 받지 않습니다. 내장된 `/code-review` 스킬이나 GitHub 용 Code Review 를 사용할 수 있습니다.

개별 결과가 기준에 미치지 못할 때는 그 개별 문제를 고치는 데서 멈추지 말고, 향후 모든 반복 작업에서 시스템 자체를 개선할 수 있도록 이를 인코딩하려고 시도하세요.

토큰 사용량 관리하기

토큰 사용량을 관리하려면 루프에 명확한 경계가 있어야 합니다:

- **작업에 맞는 기본 기능과 모델을 선택하세요:** 작은 작업에는 여러 에이전트나 루프가 필요하지 않습니다. 일부 작업은 더 저렴하고 빠른 모델을 사용할 수 있습니다.
- **명확한 성공 및 정지 기준을 정의하세요:** 완료가 무엇인지 구체적으로 밝혀 Claude 가 (너무 빠리는 아니지만) 더 빨리 해법에 도달할 수 있도록 하세요.
- **대규모 실행 전에 파일럿을 진행하세요:** 동적 워크플로는 수백 개의 에이전트를 생성할 수 있습니다. 먼저 작은 규모로 사용량을 가늠해 보세요.
- **결정론적인 작업에는 스크립트를 사용하세요:** 스크립트를 실행하는 것이 단계별로 추론하는 것보다 저렴합니다. 예를 들어 PDF 스킬은 매번 코드를 새로 도출하는 대신 Claude 가 실행할 수 있는 양식 채우기 스크립트를 함께 제공할 수 있습니다.

- **필요 이상으로 루틴을 자주 실행하지 마세요:** 감시 대상이 얼마나 자주 바뀌는지에 맞춰 간격을 설정하세요.
- **사용량을 검토하세요:** `/usage` 명령은 최근 사용량을 스킬, 서브에이전트, MCP 별로 분류해 보여주고, 인자 없이 실행한 `/goal`은 지금까지의 턴 수와 토큰 사용량을 보여주며, `/workflows`는 각 에이전트의 토큰 사용량을 보여주고 언제든지 에이전트를 중단시킬 수 있습니다.

모델과 effort 레벨 선택은 루프 비용을 좌우하는 가장 큰 요인 중 하나입니다.

시작하기

요약하면 다음과 같습니다:

루프	사용자가 넘기는 것	사용 시점	활용 기능
턴 기반	검증 과정	탐색하거나 판단을 내리는 중일 때	커스텀 검증 스킬
목표 기반	정지 조건	완료 상태가 무엇인지 알고 있을 때	<code>/goal</code>
시간 기반	트리거	작업이 프로젝트 밖에서 일정에 따라 일어날 때	<code>/loop</code> , <code>/schedule</code>
능동형	프롬프트 자체	작업이 반복적이고 명확하게 정의되어 있을 때	위의 모든 것 + 동적 워크플로

루프를 시작하려면 지금 하고 있는 작업을 살펴보고, 사용자 자신이 병목이 되는 작업을 하나 골라 어떤 부분을 넘길 수 있을지 물어보세요. 검증 체크를 직접 작성할 수 있나요? 목표가 충분히 명확한가요? 작업이 일정에 따라 들어오나요?

아이디어가 생겼다면 루프를 실행해 보고, 어디서 멈추는지 혹은 어디서 과도하게 개입하는지 결과를 관찰한 뒤, 주저하지 말고 반복해서 개선해 나가세요.

더 자세한 내용은 병렬로 에이전트를 실행하는 방법에 대한 Claude Code 문서와, loop, schedule, goal, 동적 워크플로 관련 문서 페이지를 참고하세요.

이 글은 Delba de Oliveira와 Michael Segner가 작성했습니다.